

Міністерство освіти і науки України
Черкаський національний університет імені Богдана Хмельницького
Факультет обчислювальної техніки, інтелектуальних та управляючих систем
Кафедра програмного забезпечення автоматизованих систем

КУРСОВА РОБОТА З ОБ'ЄКТНО-ОРІЄНТОВАНОГО
ПРОГРАМУВАННЯ

на тему «гра Шахи»

Студента 2 курсу, групи КС-231
спеціальності 121 “Інженерія
програмного забезпечення”

Попов А. А.

Керівник: старший викладач,
кандидат фізико-математичних наук
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Бабенко С. В.

Оцінка за шкалою:

(національною, кількість балів, ECTS)

Члени комісії: _____

(підпис)

Порубльов І. М.

(прізвище та ініціали)

Супруненко О.О.

(прізвище та ініціали)

Гребенович Ю. Є.

(прізвище та ініціали)

Черкаси – 2025 рік

ЗМІСТ

ВСТУП.....	3
РОЗДІЛ 1. БІБЛІОТЕКА SFML ТА ЇЇ ВИКОРИСТАННЯ ДЛЯ ВІДОБРАЖЕННЯ ОБ’ЄКТІВ В ПРОГРАМНОМУ ПРОДУКТІ	4
1.1. Огляд гри «Шахи».....	4
1.2. Мультимедійна бібліотека SFML для візуалізації гри.....	4
Висновки до першого розділу.....	5
РОЗДІЛ 2. ПРОЄКТУВАННЯ ГРИ «ШАХИ».....	6
2.1. Ініціалізація позицій фігур на шахівниці.....	6
2.2. Основні класи у грі.....	7
2.3. Обробка ходів фігури та їх поведінка в різних станах гри.....	8
2.4. Об’єктна структура гри «Шахи»	13
Висновки до другого розділу.....	16
РОЗДІЛ 3. РЕАЛІЗАЦІЯ ГРИ.....	17
3.1. Взаємодія класів та об’єктів між собою.....	17
3.2. Релізація рокіровки.....	21
3.3. Реалізація проведення пішака.....	23
Висновки до третього розділу.....	24
ВИСНОВОК	25
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	26
ДОДАТКИ	27
Додаток А. Алгоритм фільтрації при оголошеному шахі	27
Додаток Б. Загальна діаграма класів для гри «Шахи»	28

ВСТУП

Шахи — стародавня стратегічна покрокова інтелектуальна гра, що має багатовікову історію. На сьогодні, ця гра є одною з найбільш поширених настільних ігор у світі. Шахи сприяють становленню особистості, вчить логічно мислити і планувати свої дії, розвиває концентрацію уваги. Також їх використовують для моделювання систем штучного інтелекту.

Метою курсової роботи полягає у розробка однокористувацької 2D-гри «Шахи» із використанням принципів об'єктно-орієнтовного програмування за допомогою мультимедійної бібліотеки SFML на мові C++. Реалізація базових механік переміщення фігур та рокування. Також в грі реалізовані принципи логіки «Мат» та «Нічия», що і є логічним завершенням гри.

Актуальність теми полягає у необхідності розробки програмного продукту як в навчальних, так і в розважальних цілях.

Завдання на виконання курсової роботи:

1. Реалізація алгоритмів та правильних підходів для розміщення фігур на гральній дошці з використанням мультимедійної бібліотеки SFML.
2. Створити алгоритми для взаємодії користувача з об'єктами та алгоритми для взаємодії гральних фігур між собою.
3. Реалізувати алгоритм пошуку допустимих ходів для кожної фігури.
4. Спроекувати об'єктну частину програми разом з об'єктами на гральній дошці в грі «Шахи».
5. Зробити висновки щодо реалізації ігрових механік класичної гри «Шахи».

РОЗДІЛ 1. БІБЛІОТЕКА SFML ТА ЇЇ ВИКОРИСТАННЯ ДЛЯ ВІДОБРАЖЕННЯ ОБ’ЄКТІВ В ПРОГРАМНОМУ ПРОДУКТІ

1.1. Огляд гри «Шахи»

Шахи — це стратегічна покрокова гра для двох осіб, в яку грають на спеціальній дошці — Шахівниці. Сама дошка поділена на 64 чорно-білих клітинки. Кожен з гравців має по 16 фігур чорного або білого кольору. Кожна з фігур має своє унікальне переміщення, яке вона може здійснити під час гри. За правилами, гру починають білі фігури. Фігури одного кольору мають право бити фігури іншого, якщо обраний хід є допустимим.

Основна суть гри — поставити «Мат» своєму опоненту, тобто створити таку ситуацію, коли король суперника буде атакований(буде зроблений «Шах») і гравець ніяк не зможе захистити короля. Також гра може завершитись тоді, коли в обох гравців немає допустимих ходів. Таке завершення гри називається нічиєю(або «Пат»). Нічия може бути оголошена з ініціативи обох гравців, якщо, наприклад, два гравця не бачать суті продовження гри.

На теренах інтернету існує досить багато аналогів шахів, в яких незмінна логіка гри, але можуть бути різні візуальні оформлення та додатковий функціонал. На деяких сайтах з грою може бути присутній підрахунок переваги над противником в «побитих» фігурах. На більш просунутих платформах, може існувати історія зіграних партій, анімації при завершенні гри, аналіз зіграної партії тощо.

1.2. Мультимедійна бібліотека SFML для візуалізації гри

Для реалізації нескладної двовимірної графіки було обрано бібліотеку SFML[6]. Виходячи з того, що для реалізації гри не потрібно використовувати

фізику або роботу з просунутою графікою (шейдери, променеве трасування тощо), було і обрано бібліотеку, яка гарно підходить для виконання поставленої задачі[1].

Була звернута увага на декілька варіантів ігрових рушіїв, написаних на мові C++, такі як Unreal Engine, Godot(з підтримкою C++)[3], проте такі варіанти виявились недоцільними через простоту проєкту та високу ресурсоемність згаданих ігрових рушіїв.

Також розглядалась бібліотека SDL[2], але, на відміну від неї, SFML розроблялась з урахуванням об'єктно-орієнтованого підходу на C++, тому і була обрана, як інструмент для розробки програмного продукту.

Висновки до першого розділу

Оскільки для реалізації простої двовимірної гри не потрібна просунута графіка або використання фізики, для розв'язання поставленої задачі було прийняте рішення використовувати бібліотеку SFML. Вона гарно підходить для роботи з 2D-графікою і дає змогу швидко реалізувати базову графіку, обробку подій, та інші необхідні елементи гри. Також, бібліотека підтримує об'єктно-орієнтовний підхід, що значно спрощує структурування коду.

РОЗДІЛ 2. ПРОЕКТУВАННЯ ГРИ «ШАХИ»

2.1. Ініціалізація позицій фігур на шахівниці

Жодна гра не вважається повноцінною без взаємодії з користувачем. У SFML для реалізації такої взаємодії передбачено клас Event[4], який дозволяє обробляти різноманітні події — натискання клавіш, кліки миші, закриття вікна тощо. Завдяки цьому механізму можна реалізувати логіку реакції гри на певні дії користувача, в нашому випадку — взаємодія з фігурами на шахівниці за допомогою лівої кнопки миші.

Проте, перш ніж реалізовувати взаємодію з фігурами, їх потрібно коректно розтавити по місцях на шахівниці. Щоб це зробити, спершу необхідно дізнатись координати(х;у) самої дошки на екрані(тобто від якого пікселя починається поле), та розмір однієї клітинки на дошці. Позиція обчислюється за формулою 2.1.

$$\text{pos} = \text{board_position} + \text{cell_size} * \text{figure_position} \quad (2.1)$$

За допомогою цих двох параметрів, можна пройти по всій шахівниці від (0;0) до (7;7) за допомогою вкладених циклів.

При натисканні лівої клавіші миші, зберігаються координати (х,у) в пікселях, тієї позиції, куди ми натиснули. Для зручності подальших дій та обчислень, треба конвертувати отримані координати в позицію на шахівниці. Блок-схема алгоритму зображена на рис. 2.1.

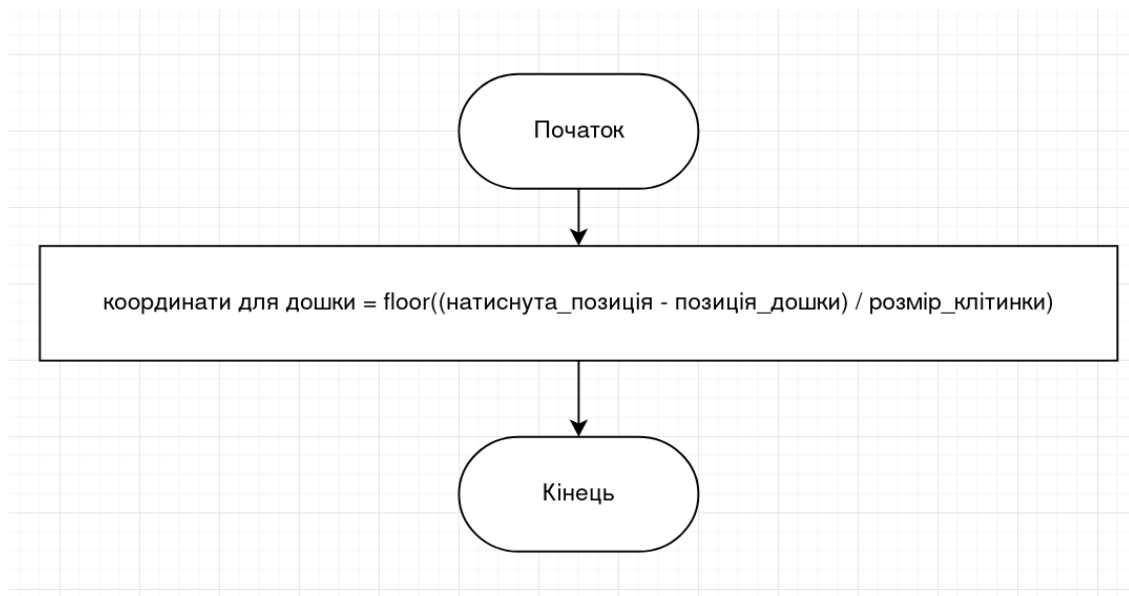


рис. 2.1 Алгоритм конвертування натиснутої позиції

Таким чином, при натисканні лівої клавіші миші по шахівниці, піксельні координати курсора конвертуються у координати клітинки дошки.

2.2. Основні класи у грі

Програма повинна мати головний клас Game, де буде відбуватись вся логіка гри: обробка подій у програмі, рокіровка, ініціалізація фігур ігрового поля тощо. Клас буде отримувати усі можливі ходи від обраної фігури, та обробляти їх, в залежності від ігрової події та позиції фігури. Також клас буде відповідати за відображення усіх об'єктів у грі та їх видалення(коли одна фігура б'є іншу, вона видаляється з гри). Окрім цього, у класі Game повинно бути реалізоване відображення підказок для можливого ходу обраної фігури.

Усі фігури повинні успадковувати батьківський клас Figure, який буде містити текстуру, позицію, розмір, тип та колір фігури. Також клас містить універсальні методи які використовуються для всіх фігур, окрім методу пошуку ходів.

Від класу Figure будуть наслідуватись фігури: Rook(тура), King(король), Queen(королева), Bishop(слон), Knight(кінь), Pawn(пішак). Кожна фігура буде містити свій метод для знаходження можливих ходів. Всі інші методи та властивості для фігур будуть універсальними, та будуть реалізовані в класі Figure.

Всі ініціалізовані фігури в класі Game будуть заноситись у структуру даних vector для подальшого використання.

2.3. Обробка ходів фігури та їх поведінка в різних станах гри

Реалізація алгоритму обробки ходів обраної фігури може працювати по різному — усе буде залежати від стану гри. Якщо гравець зробив хід та не оголосив шах або мат, то фільтрація ходу обраної фігури користувача буде виконуватись наступним чином: натиснувши на фігуру, буде викликатись метод, який повертає можливі ходи обраної фігури та зберігає їх у масив, включно з клітинками, де стоять союзні фігури.

Далі викликається метод фільтрації знайдених ходів. Він проходиться по масиву, та перевіряє кожен з ходів: виконується перебір усіх фігур виключно того ж кольору, та перевіряється їх позиція. Якщо хід, який обробляється, не збігається з жодною позицією союзної фігури, то цей хід вважається допустимим і зберігається в окремий масив, який поверне метод.

Блок-схема алгоритму звичайної фільтра зображена на рис. 2.2

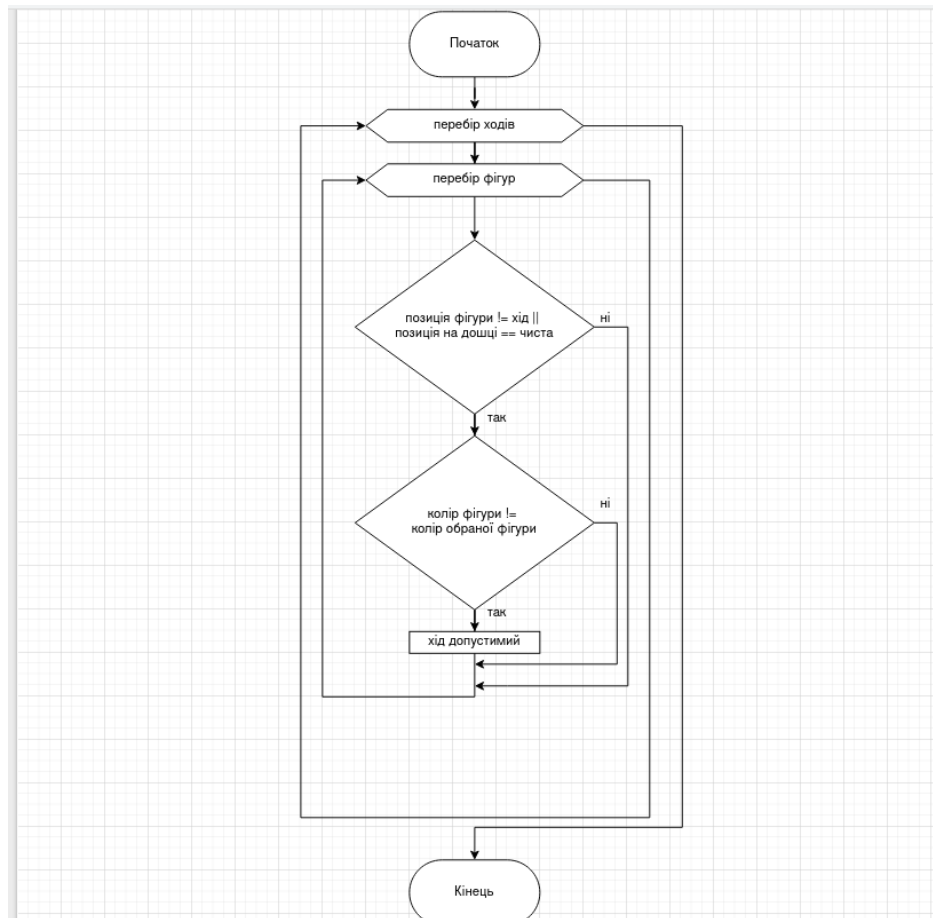


Рис. 2.2. Алгоритм фільтрації ходів
при звичайному стані гри

Інший випадок, коли після зробленого ходу, гравцю був оголошений шах. Тоді фільтрація ходу для обраної фігури буде реалізована дещо іншим способом. Враховуючи те, що обрана фігура може мати можливість захистити короля від нападу, вона буде мати можливість зробити хід лише у тому випадку, якщо цей хід призводить до знищення фігури, що атакує короля або блокує шлях атаки. Якщо фільтрація ходу поверне порожній масив, вибір фігури скидується, та гра буде чекати, поки користувач не обере іншу фігуру.

Для реалізації цього алгоритму спочатку викликається метод, який дістає короля з масиву усіх фігур, якому оголосили шах. Він зберігає

посилання на знайденого короля в змінну, для подальшого використання.

Після цього, викликається метод, який пройдеться по масиву усіх фігур, щоб перевірити скільки фігур могло напасти на фігуру короля, і, якщо таких фігур декілька, то метод повертає пустий масив, тому що подвійний шах одна фігура ніяк не зможе перекрити, можливий хід має виконати лише король, якщо він є.

Якщо, все-таки, напала всього одна фігура, викликається метод, який дістає можливі ходи фігури яка напала, та можливі ходи фігури, яка була обрана. Вкладений цикл буде перебирати кожний хід нападаючої фігури для обраної, при цьому, якщо ходи під час ітерацій співпадуть, вони занесуться в новий масив, який поверне метод.

Може бути випадок, коли програма не знаходить можливих ходів під час шаху ні для одної фігури. Тоді стан гри змінюється на мат, що і є завершенням гри, та перемогою для одного з гравців.

В алгоритм такої події повинен перебрати кожну фігуру, яка потрібного кольору, та перевірити її можливі ходи. Якщо жодна фігура не буде мати допустимих ходів, стан гри змінюється на мат.

Блок-схема алгоритму визначення мату зображена на рис. 2.3.

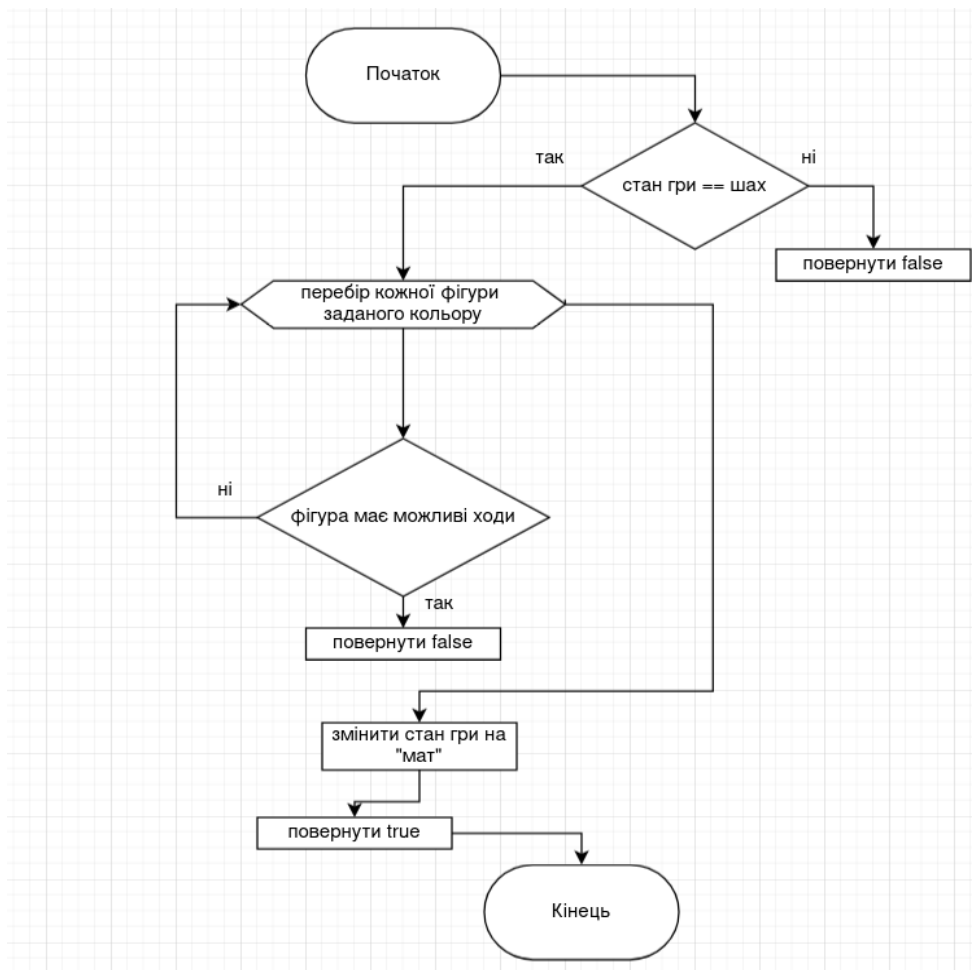


Рис. 2.3. Алгоритм визначення мату

Схожою до ситуації з матом є нічия. Вона може відбутися тоді, коли викликаний метод не знаходить можливих ходів для кожної фігури, але при цьому шах не був оголошений. Також, нічия оголошується тоді, коли на ігровому полі залишилось лише дві фігури: два королі. В таких випадках, стан гри змінюється на нічию, що також є завершенням гри.

Блок-схема алгоритму визначення нічії зображена на рис. 2.4.

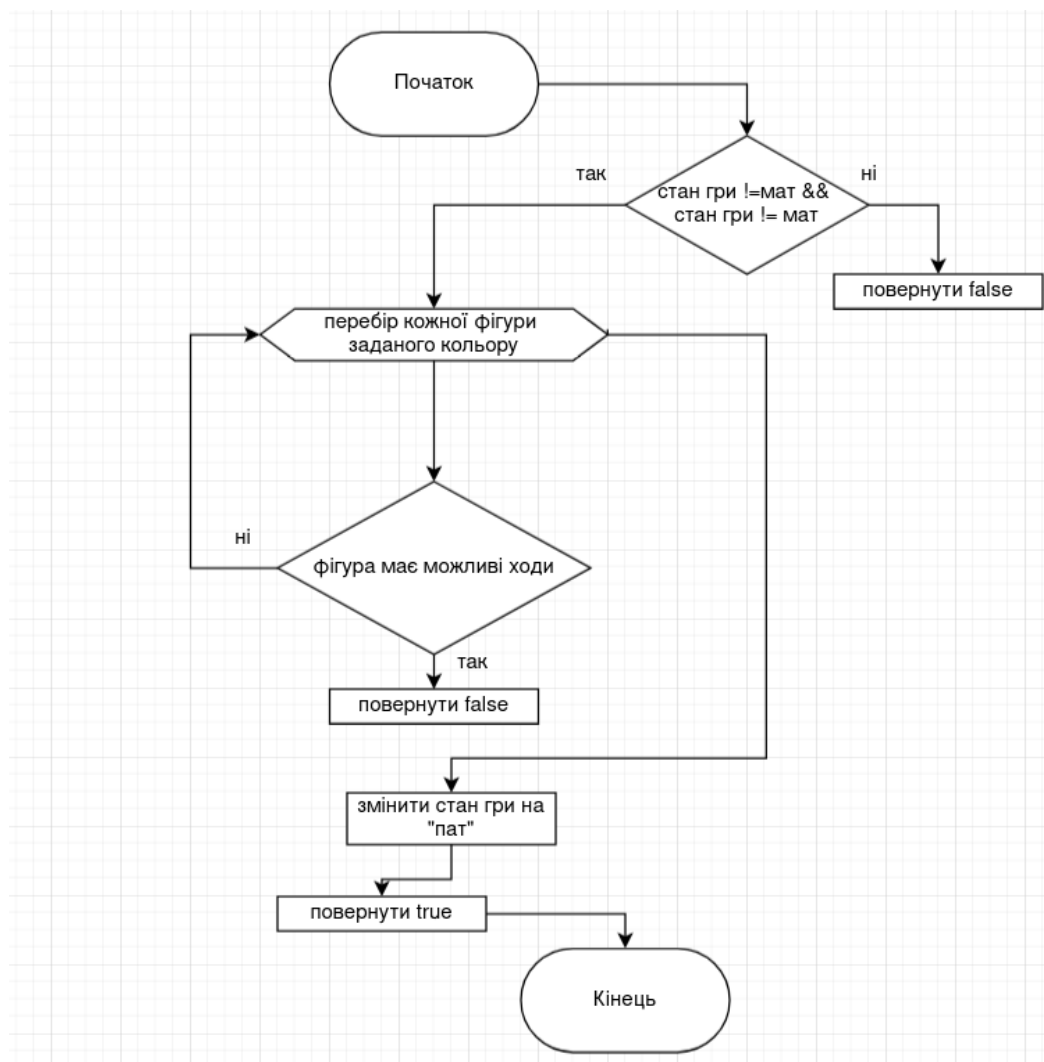


Рис. 2.4. Алгоритм визначення нічий

2.4. Об'єктна структура гри «Шахи»

У підрозділі 2.4 розглядається об'єктна структура розроблених класів для фігур, шахівниці, підказок для ходу. Об'єктна структура чітко описує власивості та методи класів.

На рис. 2.5. зображено діаграму класів ієрархії фігур. Клас Figure, як об'єкт, в грі використовуватись не буде. Figure буде успадковувати кожна із фігур, та використовувати його методи та властивості.

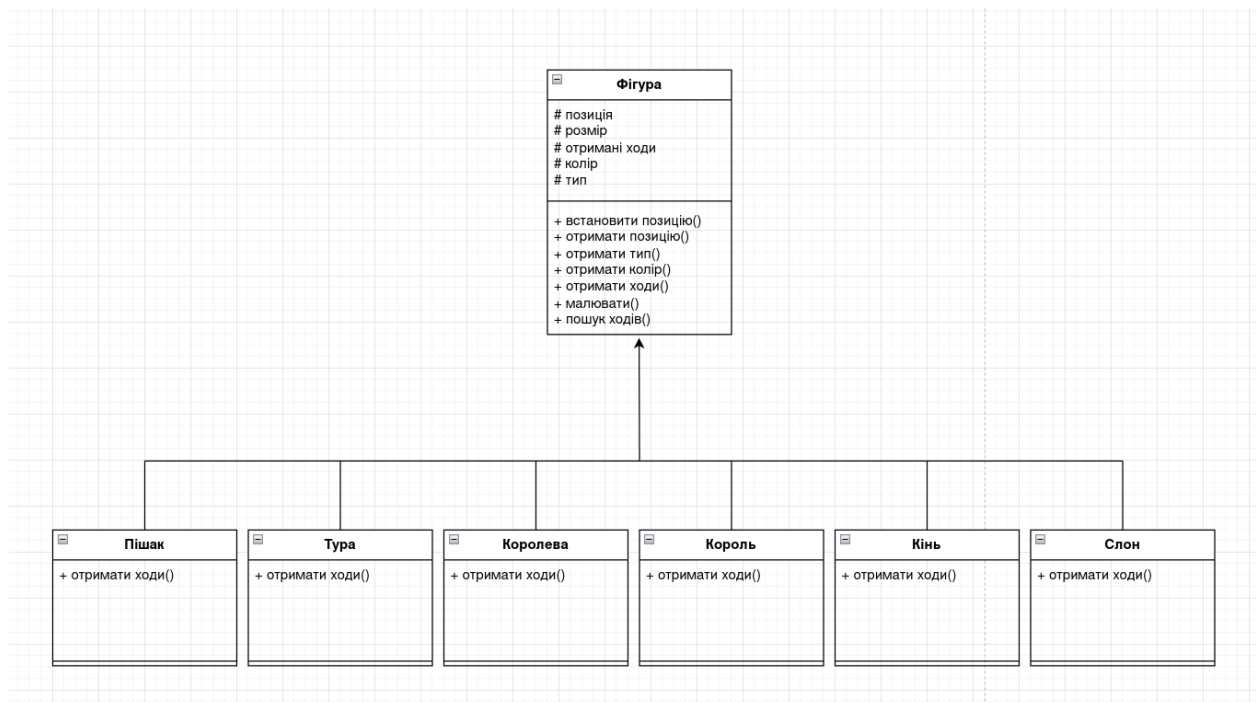


Рис. 2.5. Діаграма ієрархії фігур

Клас Game є головним класом, у якому описується вся логіка гри. Він містить в собі властивості збереження фігур, підказок, стану гри, події, чергування ходу тощо. Також клас містить методи, які відповідають за події, стан гри, відображення та видалення фігур з підказками, гетери та сетери. На рис. 2.6. зображена загальна діаграма для всіх класів гри.

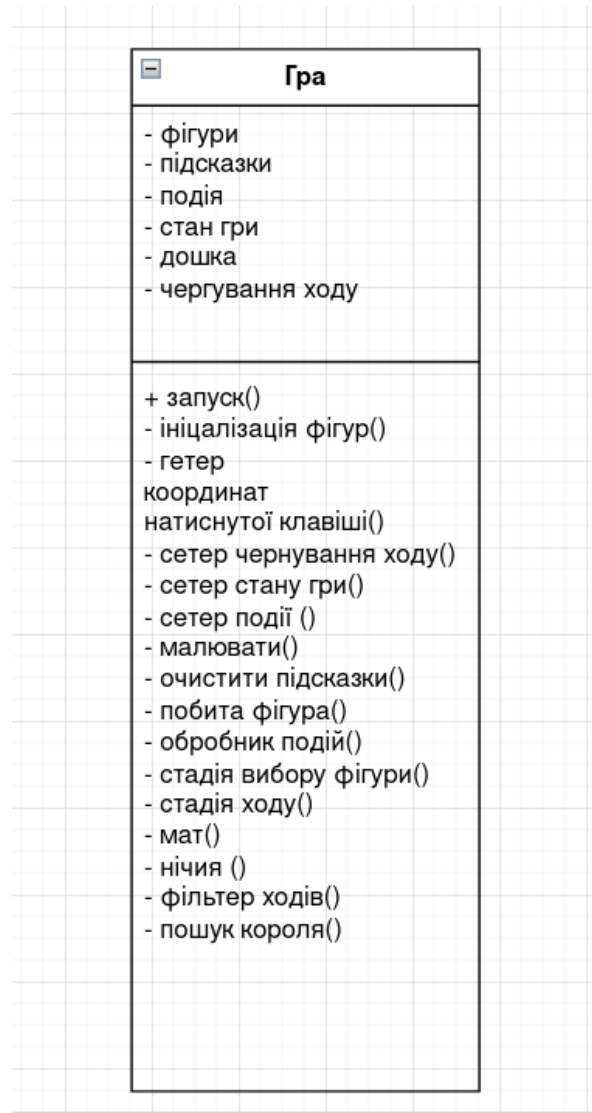


рис. 2.6 Діаграма класу Game

Клас Board відповідає за відображення шахівниці та симуляції гри у двовимірному масиві. Прототип ігрового поля дає можливість швидко відслідковувати стан конкретної позиції на полі. Це допомагає у фільтрації ходів фігур при різних станах гри, як у шахі так і в звичайному стані.

На рис. 2.7. зображена діаграма для класу Board.

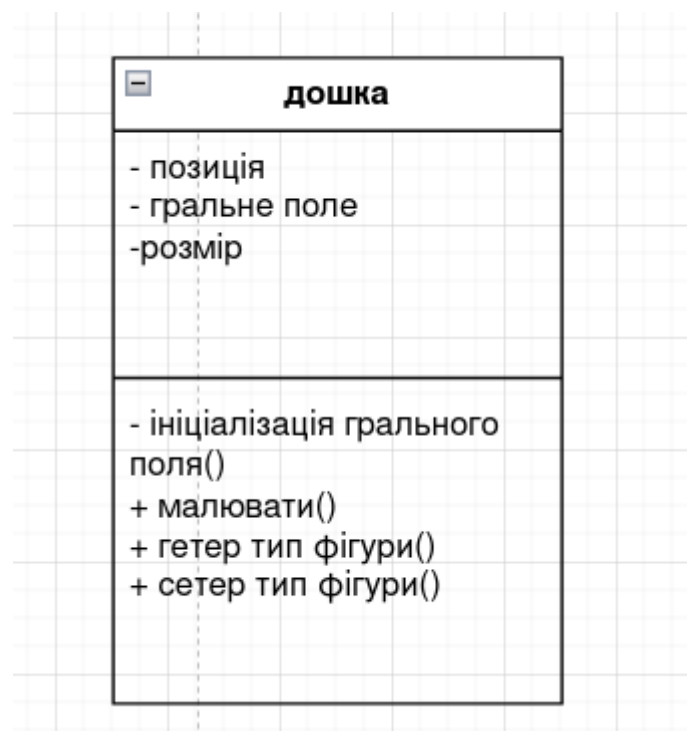


рис. 2.7. діаграма для класу Board

Один з невеликих класів, який грає важливу роль у зручності гри є клас Hint. Він дозволяє отримувати підказки ходу після стадії обрання фігури. Також клас Hint дозволяє користувачу зрозуміти, коли в грі змінюються ігрові стани: при маті, клітинка програвшого короля буде горіти червоним

напівпрозорим кольором; при нічії, клітинки обох королів загоряться сірим кольором.

На рис. 2.8. зображена діаграма для класу Hint.

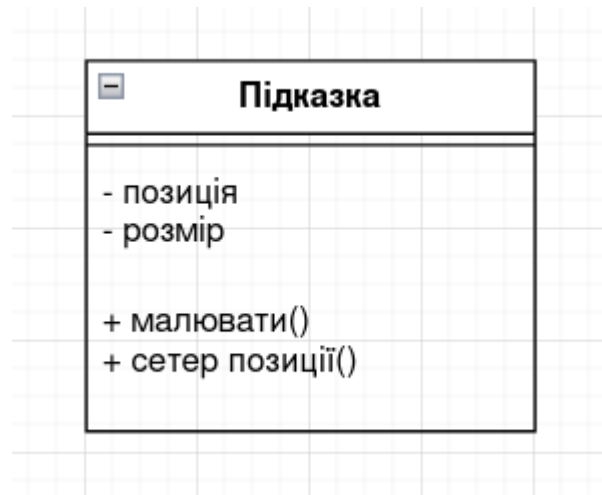


рис. 2.8. діаграма для класу Hint.

Висновки до другого розділу

У другому розділі було спроектовано архітектуру гри «Шахи». Визначено головні класи у грі (Game, Figure, Board), їх методи та взаємодію між один одним.

Розроблено логіку для різних станів гри: звичайна гра, шах, мат, нічия. Також було створено методи для фільтрації отриманих ходів для обраної фігури, в залежності від стану гри.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ГРИ

3.1. Взаємодія класів та об'єктів між собою

На початку, головний клас Game створює фігури шляхом ініціалізації об'єктів підкласів. Для зручності в подальшій роботі, всі фігури зберігаються у векторі вказівників на базовий клас Figure*. Такий підхід дозволяє керувати фігурами через єдиний інтерфейс, використовуючи механізм динамічного поліморфізму. Кожний підклас для фігури успадковується від батьківського класу Figure. Кожен підклас має однаковий метод, який виконується по різному. Він знаходить допустимі ходи для певної фігури, які потім оброблюються в класі Game.

На рис. 3.1. наведено діаграму класів фігур, які наслідують батьківський клас Figure.

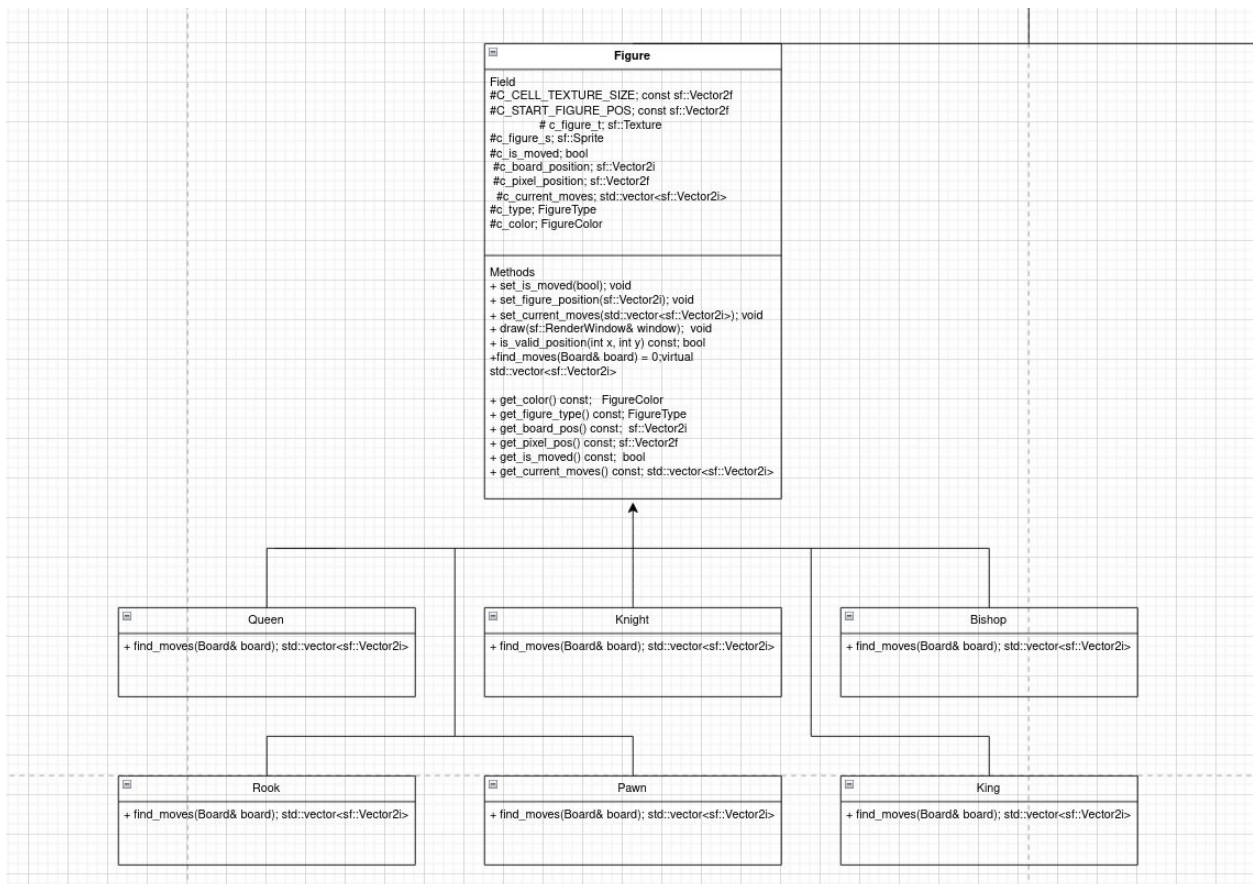


рис. 3.1. діаграма класів фігур, які наслідують батьківський клас Figure.

Разом з фігурами, клас Game також має об'єкт класу Board, який використовується для відображення шахівниці, та роботи з симуляцією ігрового поля. Доступ до масиву, у якому відбувається симуляція ігрового процесу, можна отримати тільки через методи гетерів та сетерів. Це і є прикладом інкапсуляції.

На рис. 3.2. наведено головне вікно щойно запусченої гри

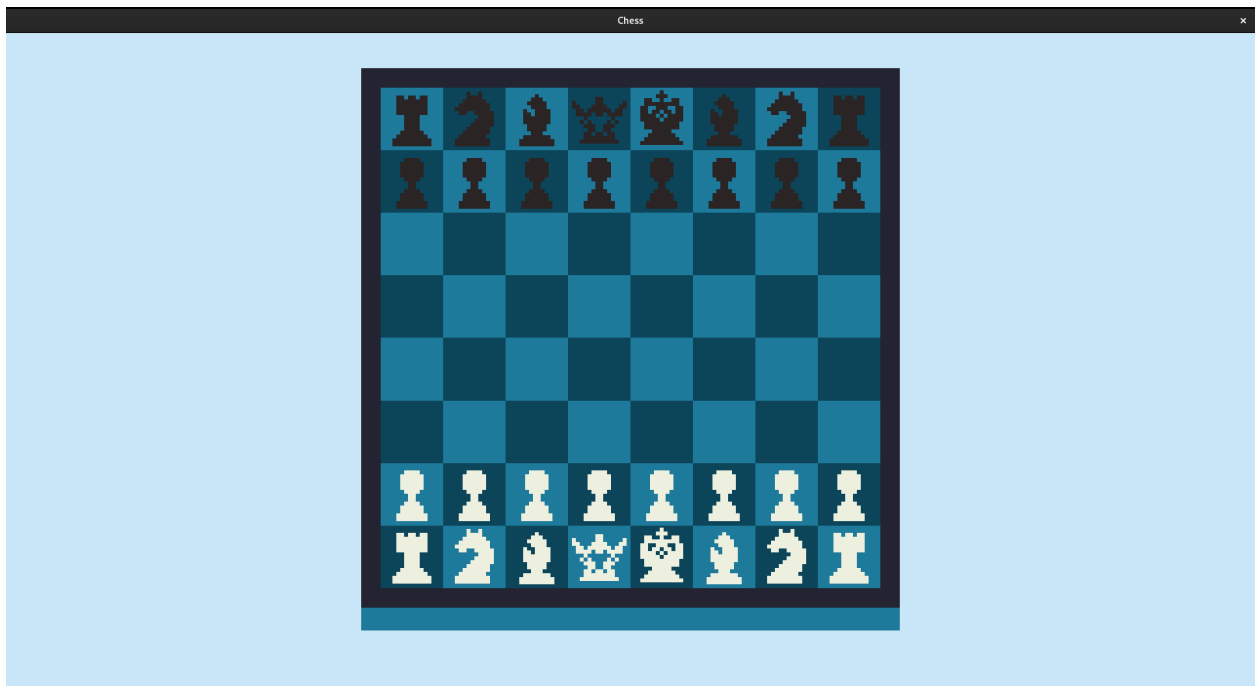


рис. 3.2. початкове вікно гри

Після вдалого запуску та ініціалізації усіх об'єктів на ігровому полі, гра переходить в очікування події. При ініціалізації конструктора класу Game, початкове значення для об'єкту enum[5] класу події(Action), було присвоєне значення «ACTION_EXPECTED», для об'єкту enum[5] класу ігрового стану(GameState) - «NORMAL». При натисканні лівої клавіші миші, викликається метод, який перевіряє належність отриманих координат до меж шахівниці. Якщо обрана клітинка містить фігуру, і саме ця фігура має право

здійснити хід(відповідно до черги), тоді, залежно від поточного стану гри, викликається метод для визначення можливих ходів цієї фігури та їх подальшої фільтрації.

Уся система координат(позиції фігур, координати можливих ходів тощо) зберігається у векторах, які є в бібліотеці SFML. Клас `sf::Vector` використовує як тип `float`, так і тип `int`. Для координації об'єктів на дошці було використано вектор типу `int(sf::Vector2i)`. Щоб змінити позицію довільного об'єкта на шахівниці, буде використовуватись формула 2.1 з другого розділу.

На рис. 3.3. метод, який приймає координати фігури на дошці, та конвертує їх в пікселі, для встановки позиції.

```
1 void Figure::set_figure_position(sf::Vector2i board_position)
2 {
3     c_board_position.x = board_position.x;
4     c_board_position.y = board_position.y;
5
6     c_pixel_position.x = C_START_FIGURE_POS.x + C_CELL_TEXTURE_SIZE.x * c_board_position.x;
7     c_pixel_position.y = C_START_FIGURE_POS.y + C_CELL_TEXTURE_SIZE.y * c_board_position.y;
8
9     c_figure_s.setPosition(c_pixel_position.x, c_pixel_position.y);
10 }
11
```

рис. 3.3. метод, який конвертує координати дошки в пікселі

Після того, як користувач успішно обрав фігурку, генерується кількість підказок, яка відповідає кількості допустимих ходів. Кожна з підказок отримує позицію допустимого ходу на шахівниці. Після ініціалізації, кожна з них заноситься у вектор підказок типу `Hint` і виводиться на екран. Підказка `hint` відображається на ігровому полі, як зелена напівпрозора клітинка. Гра переходить в очікування події «`FIGURE_PLACING`».

На рис. 3.4. Обрана фігура чекає, поки користувач здійснить хід.

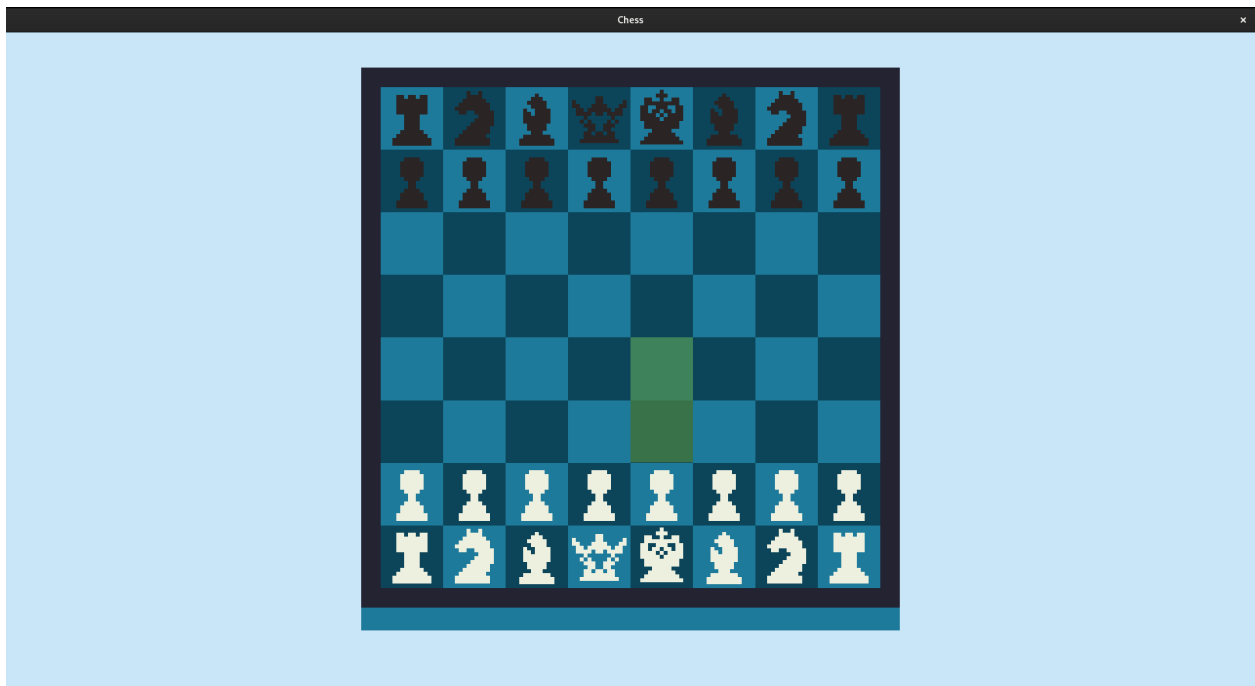


рис. 3.4. Обрана фігура чекає, поки користувач здійснить хід

Якщо гравець здійснить хід відповідно одній з підказок, здійсниться виклик методу, який очистить підказки та змінить положення значень для симуляції ігрового процесу відповідно до зробленого ходу. Фігура, яка здійснила хід, отримує нову позицію на дошці. Черга ходу зміниться, після чого виконається метод для перевірки на шах. Якщо при перевірці буде знайдена хоч одна фігура, яка напала на короля, стан гри змінюється на «Шах».

В тому випадку, якщо користувач натисне на область, де не горить зелена підказка, подія скинеться до «FIGURE_PICKING» і користувачу потрібно буде обирати фігуру по новій.

На рис. 3.5. Користувач завершив хід.

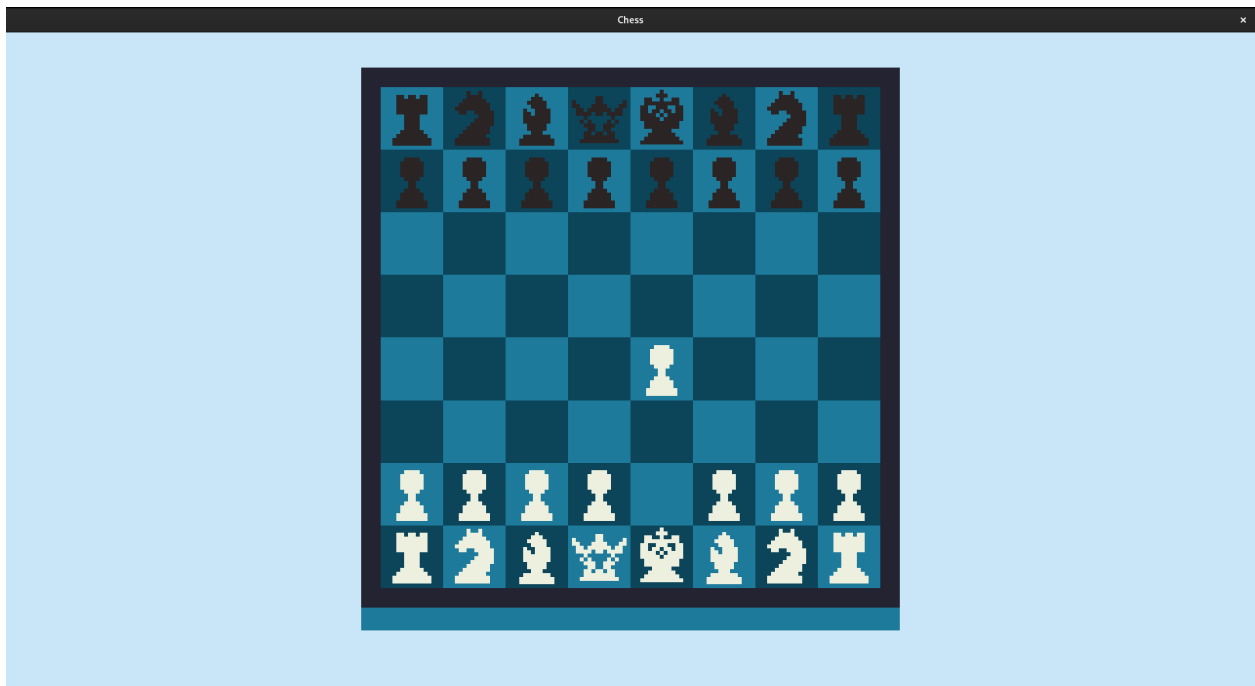


рис. 3.5. Користувач завершив хід.

3.2. Релізація рокіровки

Король зможе виконати рокіровку, лише в тому випадку, якщо будуть виконуватись наступні умови: король та одна із тур одного кольору за всю гру не зробили жодного ходу; противник не оголошував шах королю, перед моментом спроби здійснення рокіровки; весь шлях, який повинен подолати король, не буде під боєм ворожої фігури. Тільки за цих умов користувач матиме змогу здійснити одну з допустимих рокіровок.

Метод який перевіряє можливість виконати рокіровку, викликається кожен раз, коли обробляються ходи для короля, за виключенням, коли королю оголосили шах. Алгоритм методу перевіряє відразу два способи рокіровки, виходячи з відфільтрованих ходів. Спочатку перевіряється можливість короля походити на декілька клітинок по осі X вперед та назад. Якщо ходи допустимі, та королю не загрожує ворожа фігура, перевіряються тури, з

якими король може зробити рокіровку. Якщо хоча б одна з них не робила ходу, хід для короля допустимої рокіровки записується в новий масив. При виконанні події «FIGURE_PLACING» користувач може здійснити рокіровку. В такому випадку, метод перевірки на рокіровку, який викликається уже при обробці виконаного ходу, визначить напрямок (додатній або від’ємний по осі X) в який виконується рокіровка. Відповідно до цього напрямку обчислюється нова позиція тури, яка також переміщується разом із королем на відповідні клітинки.

На рис. 3.6. Користувач здійснив рокіровку.

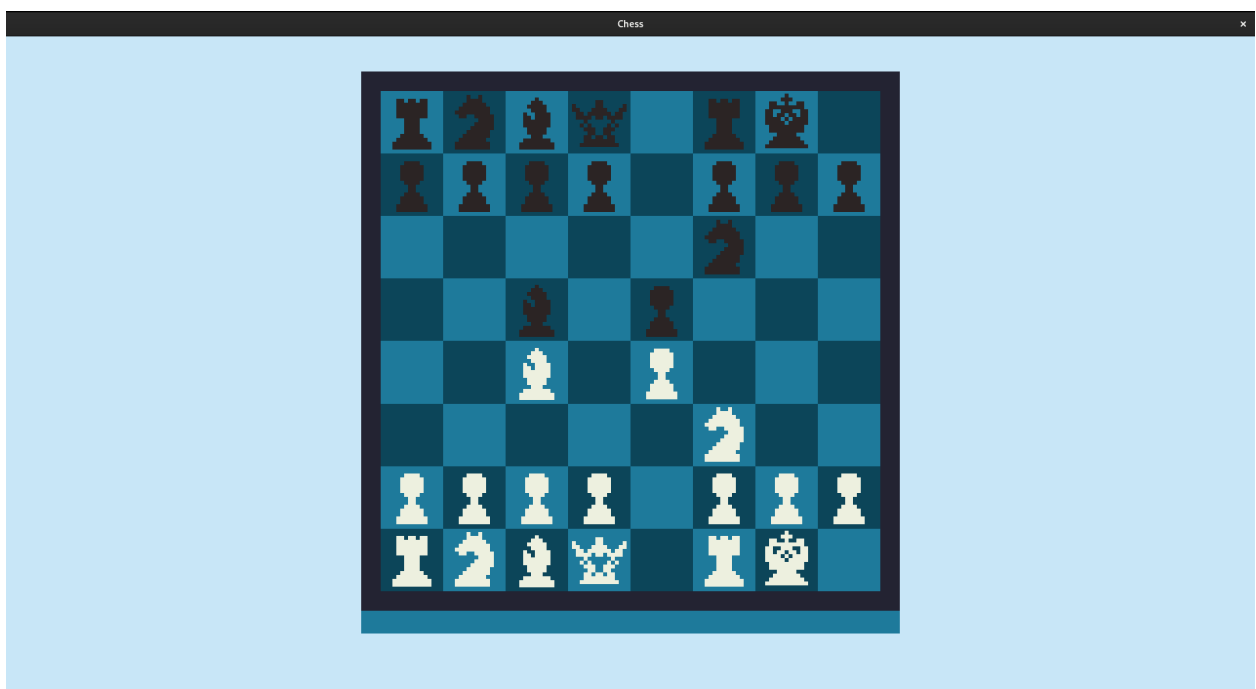


рис. 3.6. Користувач здійснив рокіровку

3.3. Реалізація проведення пішака(спрощена версія)

Проведення пішака — механізм у шахах, який дозволяє замінити проведеного пішака на іншу фігуру, якщо той дістанеться протилежної сторони дошки. У грі, ця механіка є частиною обробки події «FIGURE_PLACING».

Метод викликається після присвоєння фігурі нової позиції, але, дуже важливо, викликати метод раніше зміни черги. Реалізація алгоритму досить проста: Якщо обраною фігурою був пішак, ми перевіряємо його позицію, чи не рівна вона краю шахівниці. Якщо пішак дійшов до кінця дошки, запам'ятовуємо його позицію та видаляємо цей об'єкт. У зв'язку з тим, що ми робимо спрощену версію, замість пішака, створюємо королеву потрібного кольору, та задаємо їй позицію видаленого пішака.

На рис. 3.7. Представлений метод, який розв'язує дану задачу

```
1 void Game::pawn_to_queen()
2 {
3     if (c_define_figure->get_figure_type() == FigureType::PAWN)
4     {
5         bool is_light = (c_define_figure->get_color() == FigureColor::LIGHT) ? 1 : 0;
6         int direction = (is_light) ? 0 : 7;
7         sf::Vector2i pawn_pos = c_define_figure->get_board_pos();
8         if (pawn_pos.y == direction)
9         {
10             for(auto figure = c_figures.begin(); figure!=c_figures.end(); figure++)
11             {
12                 if ((*figure)->get_board_pos() == pawn_pos)
13                 {
14                     delete *figure;
15                     c_figures.erase(figure);
16
17                     Queen* queen = new Queen((is_light) ? LIGHT_FIGURE_QUEEN_PATH : DARK_FIGURE_QUEEN_PATH, CELL_TEXTURE_SIZE, START_FIGURE_POS,
18                     FigureType::QUEEN, (is_light) ? FigureColor::LIGHT : FigureColor::DARK);
19                     queen->set_figure_position(pawn_pos);
20                     c_figures.push_back(queen);
21                     break;
22                 }
23             }
24         }
25     }
26 }
27 return;
28 }
```

рис. 3.7. метод pawn_to_queen(), який
заміняє пішака на королеву

Висновки до третього розділу

У третьому розділі було детально розглянуто архітектуру програми, взаємодію класів між собою та реалізацію ключових механік гри, симуляцію ігрового процесу, обробку подій, генерацію підказок, визначення можливих ходів та їх обробку.

Основну логіку ігрового процесу реалізовано в класі Game, який координує роботу з фігурами, дошкою та обробкою дій користувача.

Окрему увагу було звернено на алгоритм обробки рокіровки, який враховує всі необхідні умови фігур згідно правил шахів.

ВИСНОВОК

В результаті виконання курсової роботи було спроектовано та розроблено однокористувацьку гру «Шахи» зі застосуванням бібліотеки SFML для візуалізації та управління ігровим процесом. На всіх етапах проекту застосовувались принципи об'єктно-орієнтовного програмування.

Особливу увагу було приділено правильній обробці ігрових подій, зміні станів гри та реалізації особливих шахових механік, таких як рокіровка, доведення пішака до кінця поля, фільтрація та обробка ходів для фігури, перевірка на шах, мат та нічию.

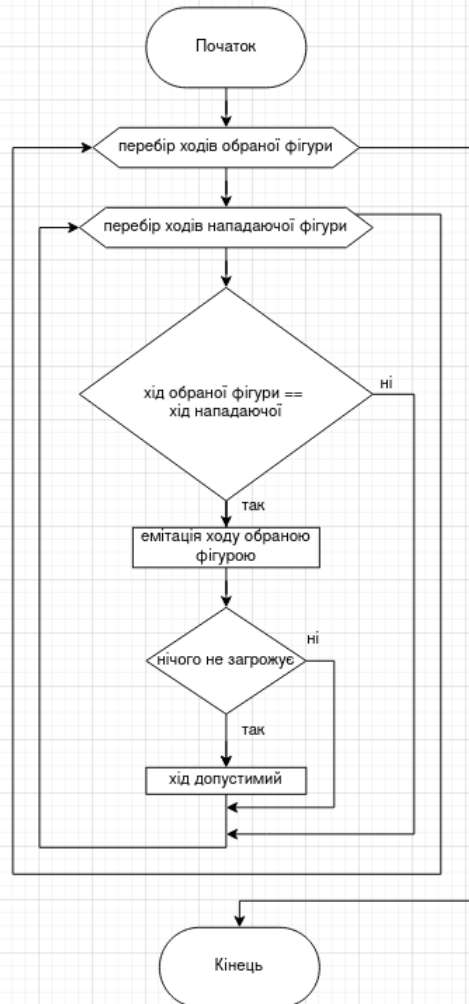
Підсумовуючи, можна запевнити, що поставлені завдання було успішно реалізовано, і створена гра повною мірою демонструє основи побудови ігрових застосунків з графічним інтерфейсом. Також у подальшому можна вдосконалити функціонал згідно з більш просунутими вимогами.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

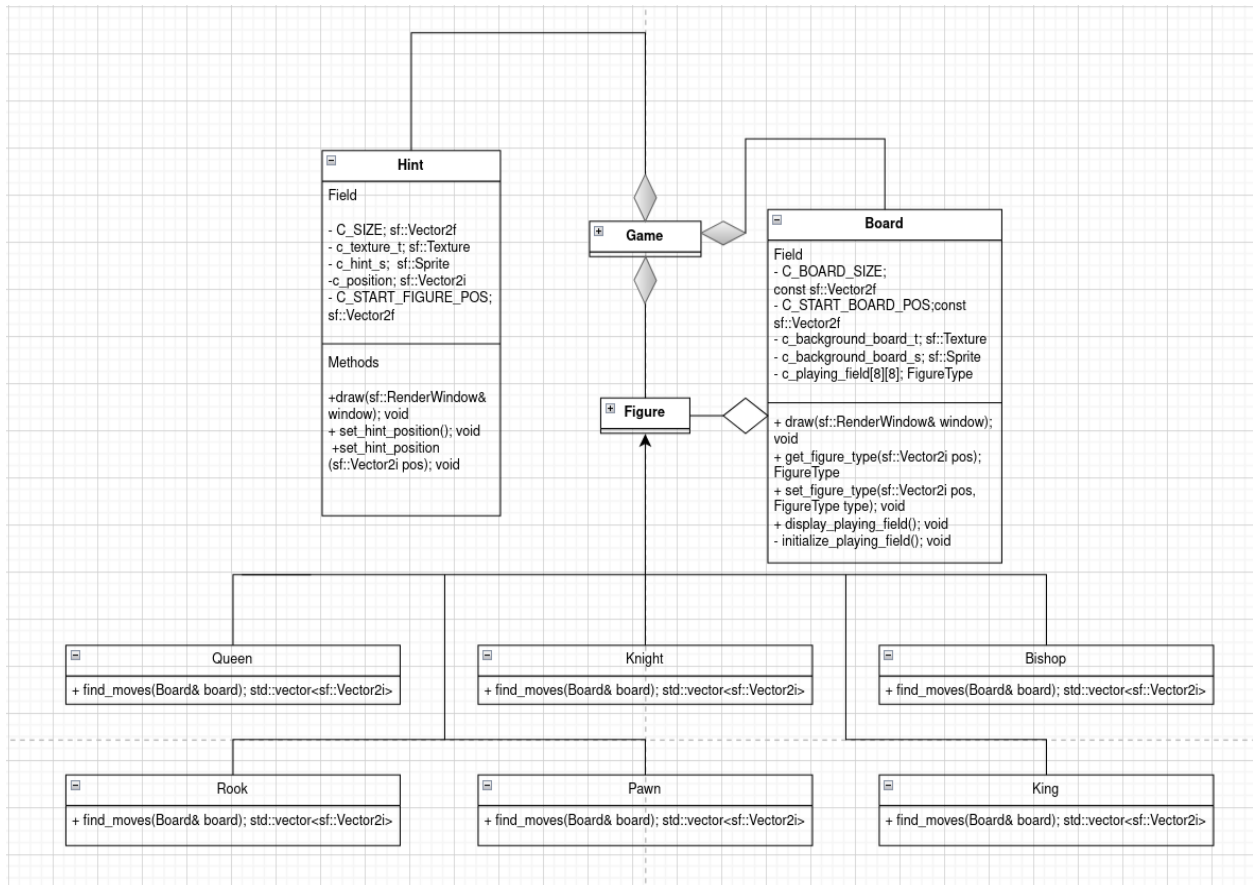
1. CHESS! // Code Review [Електронний ресурс]. Режим доступу: https://www.youtube.com/watch?v=WKs685H6uOQ&t=983s&ab_channel=TheCherno Перевірено: 26.04.25
2. SDL [Електронний документ]. Режим доступу: <https://www.libsdl.org/> Перевірено: 29.04.25
3. godot engine [Електронний документ]. Режим доступу: https://docs.godotengine.org/en/stable/contributing/development/cpp_usage_guidelines.html Перевірено: 29.04.25
4. SFML events [Електронний документ]. Режим доступу: https://www.sfml-dev.org/tutorials/3.0/window/events/#sfwindowbasehandle_events Перевірено: 12.05.25
5. Класи enum [Електронний ресурс]. Режим доступу: <https://acode.com.ua/urok-62-klasy-enum/> Перевірено: 12.05.25
6. SFML documentation [Електронний документ]. Режим доступу: <https://www.sfml-dev.org/documentation/2.6.0/> Перевірено: 26.04.25

ДОДАТКИ

Додаток А. Алгоритм фільтрації при оголошеному шахі



Додаток Б. Загальна діаграма класів для гри «Шахи»



Клас Game

Game
Field - c_window; sf::RenderWindow - c_figures; std::vector<Figure*> - c_hint; std::vector<Hint*> - c_mouse_clicked; sf::Vector2i - c_current_figure_moves; std::vector<sf::Vector2i> - c_board; Board* - c_lose; Hint* - c_draw_l; Hint* - c_draw_d; Hint* - c_state; GameState - c_action; Action - c_color; FigureColor - c_is_light_move; bool - c_define_figure; Figure*
Methods + run(); void - initialize_figures(); void - initialize_hints(); void - get_clicked_board_position(float x, float y); sf::Vector2i - get_mouse_clicked() const; sf::Vector2i - set_define_figure(Figure* figure); void - set_is_light_move(bool set); void - set_game_state(GameState state); void - set_action(Action action); void - set_figure_pos_in_playing_field(const Figure* figure, sf::Vector2i new_position); void - clear_hints(); void - figure_beated(); void - render(); void - handle_events(const sf::Event& event); void - left_mouse_clicked(); void - figure_picking(); void - figure_placing(); void - normal_state_figure_picking(); void - check_state_figure_picking(Figure* figure); void - is_check_mate(); bool - is_draw(); bool - is_current_move(std::vector<sf::Vector2i> possible_moves); bool - castling(Figure* king, std::vector<sf::Vector2i> & filtered_moves); void - castling_checker(); void - pawn_to_queen(); std::vector<sf::Vector2i> - moves_filter(std::vector<sf::Vector2i> & moves, Figure* picked_figure); - find_king(); Figure* - find_attacking_figures(const Figure* king); std::vector<Figure*> - find_protective_moves(Figure* figure); std::vector<sf::Vector2i> - is_figure_protecting(); bool

Клас Figure

